Enterprise Project for Product Companies

> I want to do a java spring boot project with next js frontend like with microservice architecture with redis, apache kafka and others so that I can put it in my resume for most of the product companies

Yes. If your goal is **placements at product companies** (Amazon, Microsoft, Atlassian, Walmart Global Tech, Adobe, Oracle, SAP, ServiceNow, JPMorgan, Goldman Sachs, Flipkart, Meesho, PhonePe, etc.), then building **one high-quality enterprise-grade project** is much more valuable than having 4–5 CRUD projects.

Since I know from our previous conversations that you're targeting **product companies** and already know **Java, Spring Boot, Next.js, MongoDB, MySQL, AWS basics, Docker, Git**, I'd recommend building something that demonstrates backend engineering skills rather than just frontend design.

The Project I'd Recommend

AI-Powered E-Commerce Platform (Enterprise Scale)

Think of it as a mini Amazon.

Not just buying products.

Include

- Authentication
- Product Catalog
- Inventory
- Order Service
- Payment
- Notifications
- Recommendation Engine
- Reviews
- Cart
- Search
- Admin Dashboard
- Analytics

using

- Java Spring Boot
- Next.js
- Microservices
- Kafka
- Redis
- Docker
- Kubernetes
- PostgreSQL
- MongoDB
- Elasticsearch
- API Gateway
- JWT
- OAuth
- CI/CD

This project alone can occupy nearly an entire interview.

Tech Stack

Backend

- Java 21
- Spring Boot 3
- Spring Security
- Spring Data JPA
- Spring Cloud
- Spring Cloud Gateway
- Spring Config Server
- Spring Eureka
- OpenFeign
- Spring Validation

Database

- PostgreSQL
- MongoDB

Use SQL where relationships matter.

MongoDB for reviews, logs, recommendations.

Cache

Redis

Use for

- Product cache
- User sessions
- OTP
- Frequently viewed products
- Rate limiting

Message Broker

Apache Kafka

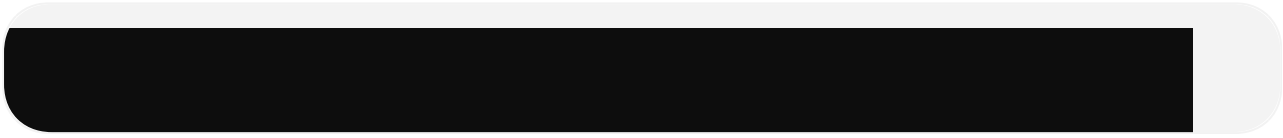
Events like

Interviewers love event-driven systems.

Search

Elasticsearch

Instead of



you'll have

- typo correction
- ranking
- filters
- autocomplete

Exactly what Amazon uses.

Storage

AWS S3

Store

- Product images
- User profile images
- Bills
- PDFs

Authentication

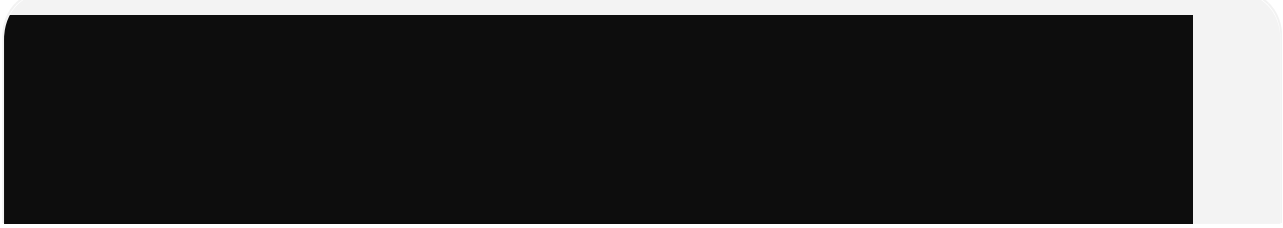
JWT

Refresh Tokens

OAuth Login

Google Login

Role Based Access



API Gateway

Spring Cloud Gateway

All requests go

Service Discovery

Eureka Server

No hardcoded URLs.

Config Server

Centralized configuration.

Docker

Every service

Kubernetes (Optional but excellent)

Deploy every microservice.

Use

- Deployment
- Service
- ConfigMap
- Secret
- Ingress

Monitoring

Prometheus

Grafana

Spring Boot Actuator

Micrometer

Logging

ELK Stack

or

Loki + Grafana

Testing

JUnit

Mockito

Testcontainers

CI/CD

GitHub Actions

Automatically

Microservices

1. User Service

Responsible for

- Register
- Login
- Profile
- Address
- JWT

Database

Postgres

---

2. Product Service

Responsible for

- Products
- Categories
- Images
- Pricing

Redis Cache

Elasticsearch

---

3. Inventory Service

Responsible for

- Stock
- Warehouse
- Quantity

Consumes Kafka

---

4. Cart Service

Redis

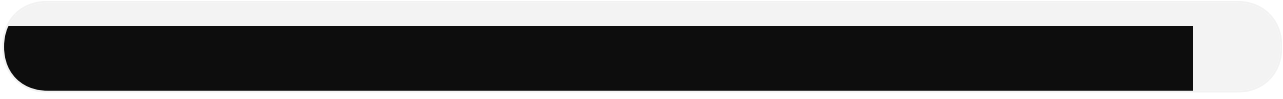
Fast operations.

---

5. Order Service

Creates orders.

Publishes



---

6. Payment Service

Fake payment gateway

or

Stripe

or

Razorpay

---

7. Notification Service

Kafka Consumer

Send

- Email
- SMS (mock)
- Push Notification

---

8. Review Service

MongoDB

Stores

Reviews

Ratings

Comments

---

9. Recommendation Service

Uses

User history

Popular products

Collaborative filtering (simple version)

10. Analytics Service

Consumes Kafka

Creates

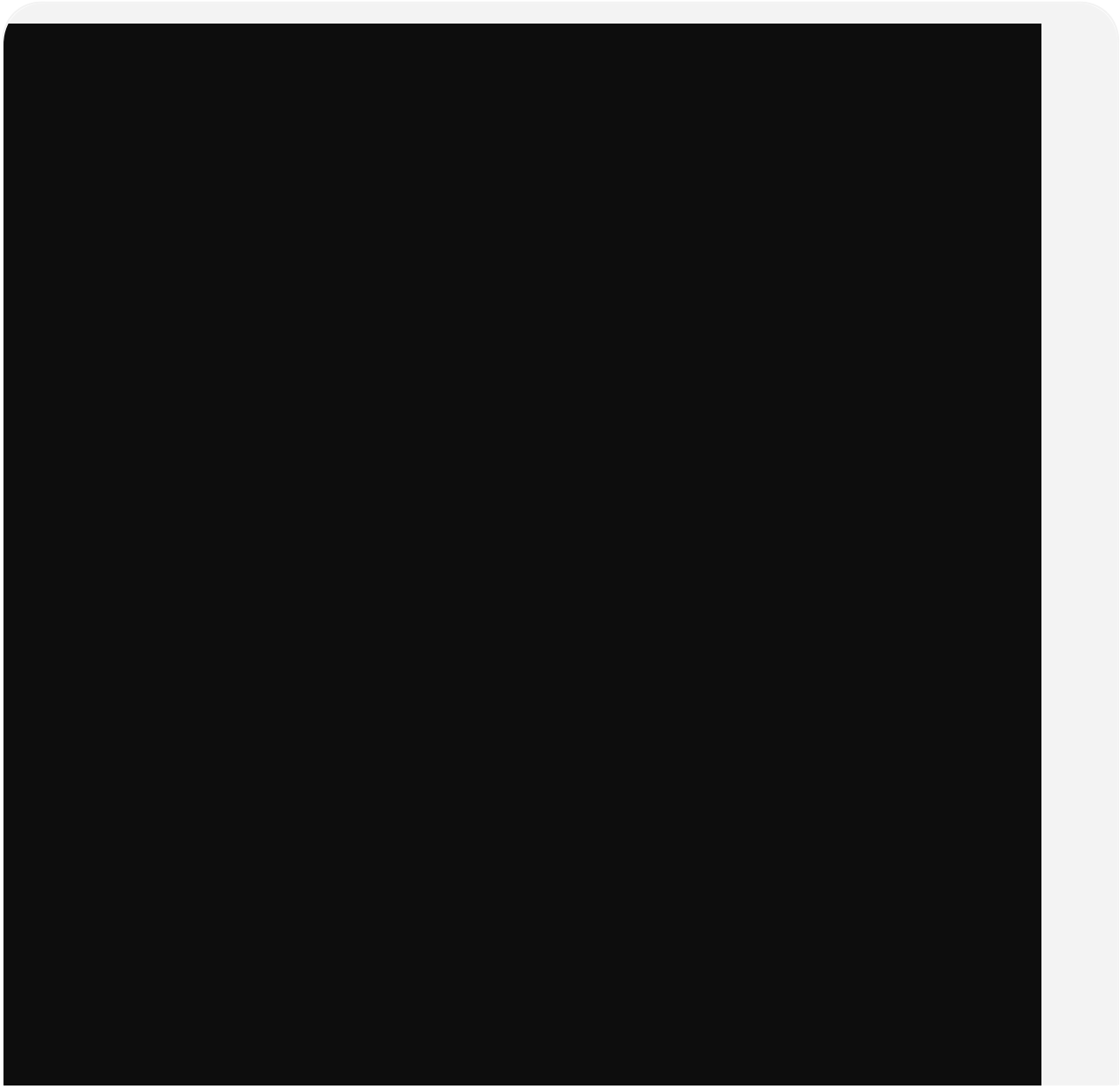
Dashboard

Sales

Revenue

Top products

Kafka Flow



Every interviewer likes this architecture.

Redis Usage

Next.js Frontend

Pages

Admin Dashboard

Can

Create Product

Delete Product

View Orders

View Users

Manage Sellers

View Revenue

Manage Coupons

Seller Dashboard

Upload Products

Sales

Inventory

Revenue

Orders

AI Features (resume standout)

Instead of just CRUD.

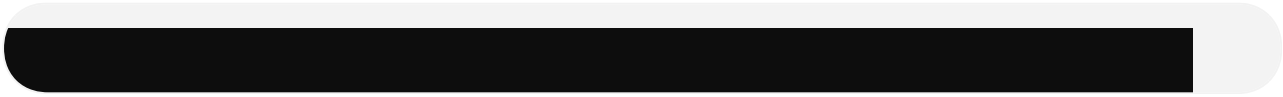
Add

AI Product Recommendation

"People who bought this also bought..."

AI Search

Natural Language



AI Review Summary

Summarize

1000 reviews

into

Pros

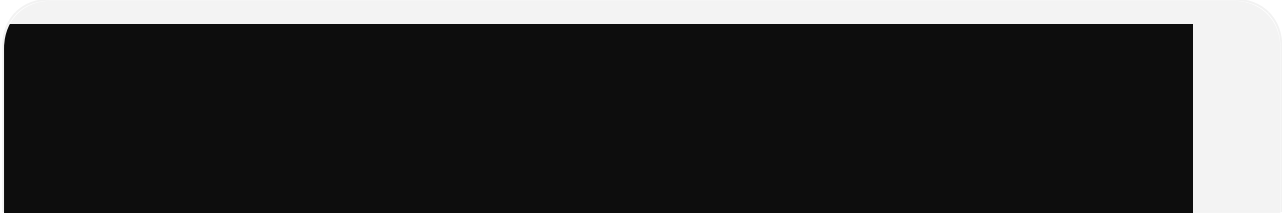
Cons

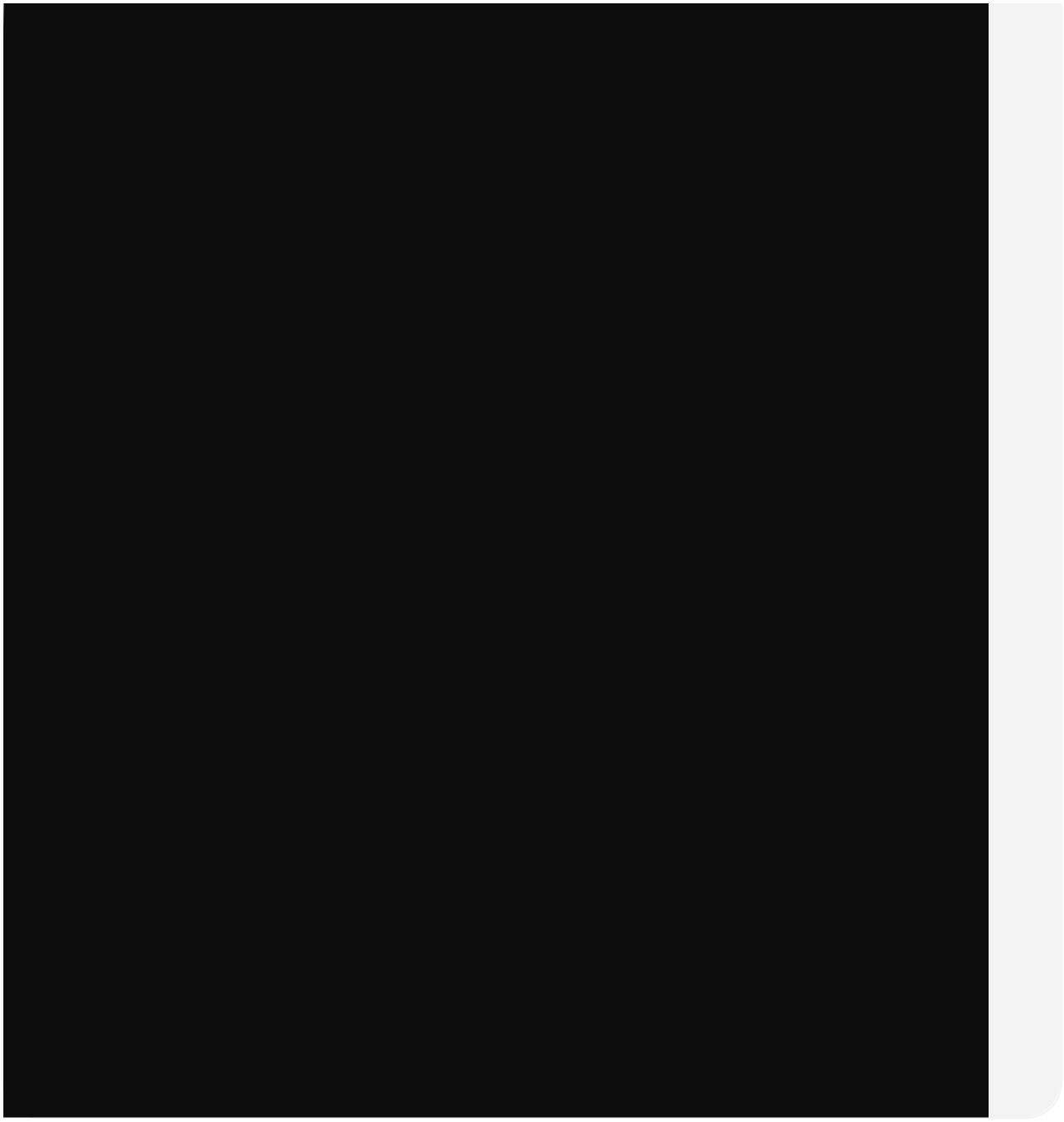
AI Chatbot

Customer Support

Using an LLM API.

Architecture





What You'll Learn

- Enterprise architecture
- REST APIs
- Security
- JWT
- OAuth
- Kafka
- Redis
- Docker
- Kubernetes

- Spring Cloud
- Caching
- Distributed systems
- Event-driven architecture
- Logging
- Monitoring
- CI/CD
- Database design
- API Gateway
- Service Discovery
- Distributed configuration

Resume Impact

A project like this signals that you understand concepts beyond basic CRUD applications. It gives you concrete examples to discuss in interviews, such as:

- Designing microservices and deciding service boundaries.
- Handling synchronous (REST/OpenFeign) versus asynchronous (Kafka) communication.
- Using Redis to reduce database load and improve latency.
- Implementing secure authentication with JWT and refresh tokens.
- Containerizing services with Docker and orchestrating them.
- Monitoring, logging, and debugging distributed systems.
- Managing data consistency across multiple services.

Suggested implementation roadmap

1. **Phase 1:** User Service + Product Service + Next.js frontend.
2. **Phase 2:** API Gateway, Eureka, Config Server, PostgreSQL, JWT authentication.
3. **Phase 3:** Cart, Order, Inventory, and Payment services.
4. **Phase 4:** Integrate Kafka and convert order processing to an event-driven flow.
5. **Phase 5:** Add Redis caching, Elasticsearch search, and Review Service.
6. **Phase 6:** Add Admin Dashboard, monitoring (Prometheus/Grafana), centralized logging, Docker Compose, and GitHub Actions CI/CD.
7. **Phase 7 (Optional):** Deploy to Kubernetes and add AI-powered recommendations, review summarization, and a customer support chatbot.

This is the kind of end-to-end, production-style project that can become the centerpiece of a resume for backend or full-stack roles at product companies. It's also

large enough that you can talk about system design, scalability, caching, messaging, and deployment during interviews—not just coding.